

Trabajo Práctico N° 2

Redes Convolucionales, Detección de Objetos y Redes Recurrentes

Deep Learning Aplicado
Tecnicatura Universitaria en Inteligencia Artificial
FCEIA - UNR

Consideraciones Generales

- Se recomienda encarecidamente guardar **checkpoints** durante el entrenamiento y aprovechar la funcionalidad de **resume** para retomar desde el último checkpoint en caso de interrupción.
- Cada ejercicio debe entregarse como un **notebook de Jupyter** (.ipynb) con celdas de código y celdas de texto (Markdown) que expliquen las decisiones tomadas, los experimentos realizados y el análisis de resultados.
- **Reproducibilidad:** Fijar una semilla aleatoria al inicio de cada experimento para que los resultados sean reproducibles. Esto incluye las semillas de Python, NumPy y el framework de deep learning utilizado:

```

1 import random, numpy as np, torch
2
3 def set_seed(seed=42):
4     random.seed(seed)
5     np.random.seed(seed)
6     torch.manual_seed(seed)
7     torch.cuda.manual_seed_all(seed)
8     torch.backends.cudnn.deterministic = True
9     torch.backends.cudnn.benchmark = False
10
11 set_seed(42)

```

- **División del dataset:** Siempre trabajar con tres conjuntos separados: **train** (entrenamiento), **val** (validación) y **test** (evaluación final). El conjunto de train es el único que el modelo ve durante el entrenamiento. El conjunto de val se usa para monitorear el entrenamiento, comparar experimentos y tomar decisiones de diseño (arquitectura, hiperparámetros). El conjunto de test se usa **una sola vez**, al final, para reportar las métricas definitivas. Usar val como criterio de selección y reportar test evita el sobreajuste implícito al conjunto de evaluación.

1. Ejercicio 1 — Clasificación de Razas de Perros con CNN desde cero

Dataset Drive: [70 Dog Breeds Image Dataset - Drive](#)

Dataset Kaggle: [70 Dog Breeds Image Dataset - Kaggle](#)

Descripción

En este ejercicio deben construir, entrenar y optimizar una red neuronal convolucional **desde cero** (sin transfer learning) para clasificar imágenes de razas de perros. El objetivo es experimentar sistemáticamente con distintas decisiones de diseño arquitectónico y de entrenamiento para maximizar el rendimiento del modelo.

Restricción: Queda **prohibido** el uso de modelos preentrenados o transfer learning en este ejercicio. Todos los pesos deben inicializarse aleatoriamente y entrenarse sobre el dataset.



Figura 1: Mosaico de imágenes de ejemplo del dataset *70 Dog Breeds Image Dataset*.

Reducción del problema (recomendado)

Dado que el entrenamiento puede ser costoso en cómputo, se recomienda trabajar con un **subconjunto de 5 a 10 razas** elegidas por el grupo. Esto permite iterar más rápido sobre los experimentos. En el informe deben justificar la selección (pueden elegir razas visualmente similares para mayor dificultad, o variadas para mayor facilidad).

Checkpoints y reanudación

Se recomienda guardar el estado del modelo periódicamente durante el entrenamiento para poder retomarlo en caso de interrupción:

```
1 # Guardar checkpoint
2 torch.save({
3     'epoch': epoch,
4     'model_state_dict': model.state_dict(),
5     'optimizer_state_dict': optimizer.state_dict(),
6     'loss': loss,
7 }, 'checkpoint_epoch_{}.pth'.format(epoch))
```

```

8
9 # Retomar desde checkpoint
10 checkpoint = torch.load('checkpoint_epoch_N.pth')
11 model.load_state_dict(checkpoint['model_state_dict'])
12 optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
13 start_epoch = checkpoint['epoch'] + 1

```

Experimentos a realizar

Deberán documentar al menos **4 experimentos distintos**, organizados en una tabla comparativa. A continuación se sugiere una progresión lógica:

Experimento	Qué modificar
E1 — Línea base	CNN simple: 3 bloques Conv-BN-ReLU-MaxPool + cabeza clasificadora densa.
E2 — Arquitectura más profunda	Aumentar profundidad. Explorar el efecto de agregar más capas convolucionales y variar el tamaño de los filtros (3×3 vs 5×5) y los strides.
E3 — Bloques avanzados	Incorporar al menos uno de los siguientes: bloques residuales (skip connections estilo ResNet), módulos Inception (ramas paralelas con filtros de distinto tamaño y convoluciones 1×1 como cuello de botella).
E4 — Regularización y Data Augmentation	Agregar Dropout, BatchNormalization si no estaba, y un pipeline de Data Augmentation (flips horizontales, rotaciones, variaciones de brillo/contraste, recortes aleatorios, etc.).

Dentro de cada experimento también pueden variar hiperparámetros como learning rate, batch size, número de épocas y optimizador.

Preparación de datos

- Aplicar **resize** a una resolución fija (ej. 128×128 o 224×224).
- **Normalizar** los píxeles (media y desviación estándar del conjunto de entrenamiento, o los valores estándar de ImageNet).
- Implementar un pipeline de **Data Augmentation** sobre el conjunto de entrenamiento (no sobre validación ni test).

Qué reportar

Para cada experimento, completar la siguiente tabla:

Experimento	Acc. Train	Acc. Val	Acc. Test	# Parámetros	Épocas	Observaciones
E1						
E2						
E3						
E4						

Además, para el **mejor modelo** obtenido:

- Curvas de loss y accuracy (train vs. validación) por época.
- **Matriz de confusión** en el conjunto de test.
- Al menos 5 ejemplos de predicciones correctas e incorrectas con la imagen, la clase real y la clase predicha.
- Análisis breve: ¿qué razas se confunden más entre sí? ¿Por qué?

2. Ejercicio 2 — Detección de Señales de Tráfico con YOLO y Transfer Learning

Dataset Drive: [Traffic Signs Detection - Drive](#)

Dataset Kaggle: [Traffic Signs Detection - Kaggle](#)

Clases (15): Green Light, Red Light, Speed Limit 10, Speed Limit 20, Speed Limit 30, Speed Limit 40, Speed Limit 50, Speed Limit 60, Speed Limit 70, Speed Limit 80, Speed Limit 90, Speed Limit 100, Speed Limit 110, Speed Limit 120, Stop.

Descripción

En este ejercicio deberán aplicar **transfer learning** sobre YOLO (Ultralytics) para detectar y clasificar señales de tráfico en imágenes reales. El objetivo es familiarizarse con el flujo completo de fine-tuning de un detector de objetos moderno: inspección del dataset, configuración, entrenamiento, y análisis exhaustivo de resultados.

2.1. Inspección y comprensión del dataset

El dataset ya se encuentra en **formato YOLO de Ultralytics**, listo para usar. Su estructura es la siguiente:

```

1 dataset/
2 |-- data.yaml          # configuracion: rutas, numero de clases y nombres
3 |-- train/
4 |   |-- images/        # imagenes de entrenamiento
5 |   +-- labels/        # anotaciones .txt (una por imagen)
6 |-- valid/
7 |   |-- images/
8 |   +-- labels/
9 +-- test/
10    |-- images/
11    +-- labels/

```

El archivo `data.yaml` contiene toda la información necesaria para el entrenamiento: las rutas a los splits, el número de clases (`nc`) y la lista de nombres de clases (`names`). **Leer y entender este archivo es el primer paso antes de entrenar.**

Cada archivo `.txt` de anotación sigue el formato YOLO: una línea por objeto, con el formato `class_id x_center y_center width height` donde todas las coordenadas están **normalizadas entre 0 y 1**.

Tarea de verificación visual

Antes de entrenar, graficar al menos 8-10 imágenes del conjunto de entrenamiento con sus bounding boxes y etiquetas superpuestas. Esto permite confirmar que el formato está siendo interpretado correctamente y que las anotaciones son coherentes con el contenido visual.



Figura 2: Ejemplo de imagen del dataset *Traffic Signs Detection* con bounding boxes y etiquetas de clase superpuestas.

```
1 import cv2
2
3 def visualizar_annotacion(img_path, lbl_path, class_names):
4     img = cv2.imread(img_path)
5     h, w = img.shape[:2]
6     colors = [(0,255,0), (0,0,255), (255,0,0), (0,255,255), (255,0,255),
7               (255,255,0)]
8
9     with open(lbl_path) as f:
10         for i, line in enumerate(f):
11             cls, cx, cy, bw, bh = line.strip().split()
12             cls = int(cls)
```



```

12         cx, cy, bw, bh = float(cx)*w, float(cy)*h, float(bw)*w, float
    (bh)*h
13         x1, y1 = int(cx - bw/2), int(cy - bh/2)
14         x2, y2 = int(cx + bw/2), int(cy + bh/2)
15         color = colors[i % len(colors)]
16         cv2.rectangle(img, (x1, y1), (x2, y2), color, 2)
17         label = class_names[cls]
18         (tw, th), _ = cv2.getTextSize(label, cv2.FONT_HERSHEY_SIMPLEX
    , 0.5, 1)
19         ty1 = max(y1 - th - 4, 0)
20         cv2.rectangle(img, (x1, ty1), (x1 + tw + 4, ty1 + th + 4),
    color, -1)
21         cv2.putText(img, label, (x1 + 2, ty1 + th + 1),
22                     cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,0,0), 1)
23         return img
24
25     names = ['Green Light', 'Red Light', 'Speed Limit 10', 'Speed Limit 100',
26             'Speed Limit 110', 'Speed Limit 120', 'Speed Limit 20', 'Speed
    Limit 30',
27             'Speed Limit 40', 'Speed Limit 50', 'Speed Limit 60', 'Speed
    Limit 70',
28             'Speed Limit 80', 'Speed Limit 90', 'Stop']
29
30     img = visualizar_anotacion(img_path, lbl_path, names)
31     cv2.imwrite(out_path, img)

```

2.2. Entrenamiento con YOLO

Instalar Ultralytics y entrenar el modelo:

```

1 # Instalacion
2 # !pip install ultralytics
3
4 from ultralytics import YOLO
5
6 # Cargar modelo preentrenado (elegir tamaño según recursos disponibles)
7 model = YOLO("yolo11n.pt") # nano: mas rapido | yololls.pt: mas preciso
8
9 # Entrenar
10 results = model.train(
11     data="ruta/al/data.yaml",
12     epochs=50,
13     imgsz=640,
14     project="tp2_senales",
15     name="v1"
16 )

```

Elección del tamaño del modelo: YOLO está disponible en variantes n (nano), s (small), m (medium), l (large) y x (extra-large). Deben entrenar y comparar **al menos dos tamaños** distintos, analizando el trade-off entre velocidad de inferencia y precisión.

2.3. Análisis de resultados

YOLO genera automáticamente un conjunto de archivos de salida en la carpeta del experimento. Deben incluir e interpretar **todos** los siguientes:

Curvas de entrenamiento

Archivo	Qué muestra
results.png	Dashboard general: pérdidas de train/val (box, cls, dfl) y métricas (precision, recall, mAP50, mAP50-95) por época. Permite evaluar convergencia y detectar overfitting.
BoxPR_curve.png	Curva Precision-Recall por clase a IoU 0.5. El área bajo cada curva es el AP50 de esa clase.
BoxF1_curve.png	Curva F1 vs. umbral de confianza. Indica el threshold óptimo para inferencia (pico de F1).
BoxP_curve.png	Precision vs. confianza por clase. Muestra cuán "limpias" son las detecciones al subir el umbral.
BoxR_curve.png	Recall vs. confianza por clase. Muestra cuántos positivos reales se pierden al subir el umbral.

Matrices de confusión

Archivo	Qué muestra
confusion_matrix.png	Matriz de confusión en conteos absolutos, incluyendo filas/columnas de fondo (<i>background</i>): permite ver falsos positivos y falsos negativos.
confusion_matrix_normalized	Igual, pero normalizada por fila. Más útil para comparar clases con distinto número de instancias.

2.4. Qué reportar

- **mAP@50, mAP@50-95**, tiempo de entrenamiento, tiempo de inferencia por imagen.
- Interpretación de todas las imágenes listadas en la sección 2.3.
- Análisis cualitativo: ¿qué clases detecta mejor el modelo? ¿Cuáles confunde? ¿Por qué creen que ocurre eso?
- Al menos 5 ejemplos de predicciones del modelo sobre imágenes del conjunto de test (con bboxes y scores visibles).

3. Ejercicio 3 — Reconocimiento de Comandos de Voz con CRNN

Dataset Drive: [Google Speech Commands v2 - Drive](#)

Dataset Kaggle: [Google Speech Commands v2 - Kaggle](#)

Clases sugeridas (10): yes, no, up, down, left, right, on, off, stop, go.

Descripción

En este ejercicio deben construir una **Red Neuronal Convolutacional-Recurrente (CRNN)** para reconocer comandos de voz a partir de grabaciones de audio de un segundo. El dataset contiene grabaciones pronunciadas por muchos hablantes distintos en condiciones acústicas reales (ruido de fondo, variaciones de micrófono), lo que hace que el modelado

temporal de la RNN tenga valor real: comandos como `down` o `stop` son secuencias fonéticas con estructura temporal que una red sin memoria no puede capturar correctamente.

La arquitectura sigue un esquema **many-to-one**: la CNN procesa cada ventana temporal del espectrograma de forma independiente y produce un vector de características; la secuencia de esos vectores entra a la RNN, que modela cómo los patrones acústicos evolucionan en el tiempo; el último estado oculto se clasifica con una capa densa.

Reducción del problema: Si el entrenamiento completo resulta demasiado pesado para los recursos disponibles, pueden trabajar con un subconjunto de **4 a 6 clases** (ej. `yes`, `no`, `stop`, `go`). Deben justificar la selección en el informe. Se recomienda guardar **checkpoints** regularmente para poder retomar el entrenamiento ante interrupciones.

Pipeline

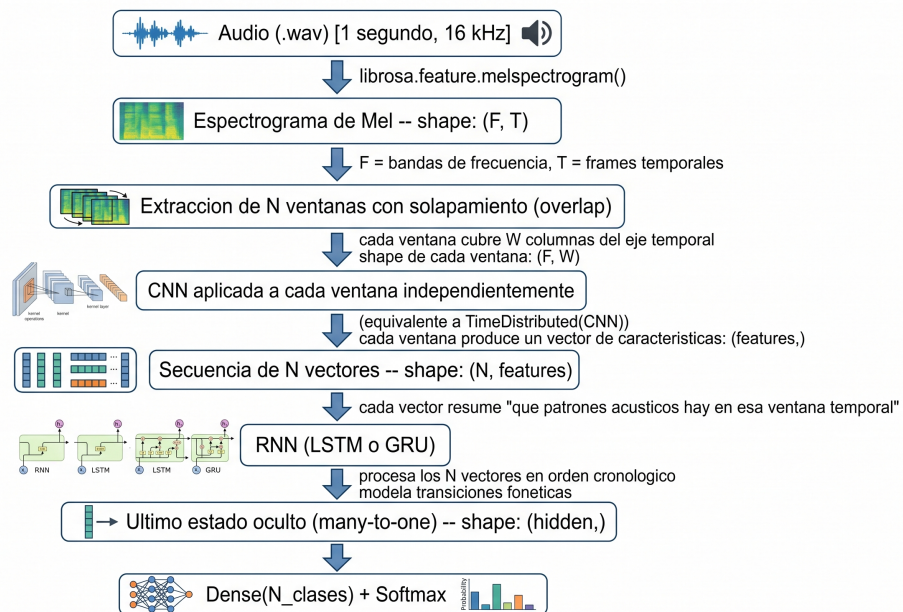


Figura 3: Pipeline propuesto para la arquitectura CRNN, desde el audio de entrada hasta la clasificación final.

3.1. Preprocesamiento: Del audio al espectrograma de Mel

Los espectrogramas de Mel representan la energía de la señal de audio en función del tiempo (eje horizontal) y la frecuencia en escala Mel (eje vertical). La escala Mel comprime las frecuencias altas y expande las bajas, imitando la percepción auditiva humana. El color representa la intensidad de energía en decibelios (dB): colores cálidos indican alta energía, colores fríos indican baja energía o silencio.

```
1 import librosa
2 import numpy as np
3
4 def audio_a_mel(path, sr=16000, n_mels=64, n_fft=1024, hop_length=512):
5     y, sr = librosa.load(path, sr=sr) # cargar y resamplear a 16
6     mel = librosa.feature.melspectrogram(
```

```

7     y=y, sr=sr, n_mels=n_mels,
8     n_fft=n_fft, hop_length=hop_length
9 )
10 mel_db = librosa.power_to_db(mel, ref=np.max) # escala logaritmica (dB
11 )
12 return mel_db # shape: (F, T) = (n_mels, T)

```

Pasos obligatorios

- Fijar $sr=16000$ (frecuencia de muestreo del dataset).
- Asegurarse de que todos los espectrogramas tengan la **misma longitud temporal T** (padding con ceros al final o truncado).
- **Normalizar** (media 0, desvío 1) calculando estadísticas **solo sobre el conjunto de entrenamiento** y aplicándolas a validación y test.

Visualización y análisis de espectrogramas (obligatorio)

Antes de entrenar, graficar al menos **8 espectrogramas**: 4 instancias de la misma clase y 4 de clases distintas. Para cada uno, describir explícitamente:

- **Eje horizontal (tiempo)**: ¿Cuánto dura la señal útil? ¿Hay silencio al inicio o al final?
- **Eje vertical (frecuencia Mel)**: ¿En qué rango de frecuencias se concentra la energía del comando?
- **Color (energía en dB)**: ¿Dónde están las zonas de alta energía? ¿Qué representan las bandas horizontales brillantes?

Luego responder: ¿pueden distinguir visualmente espectrogramas de la misma clase entre sí? ¿Y entre clases distintas (ej. yes vs no, up vs down)? ¿Qué diferencias observan en la distribución temporal y frecuencial de la energía?

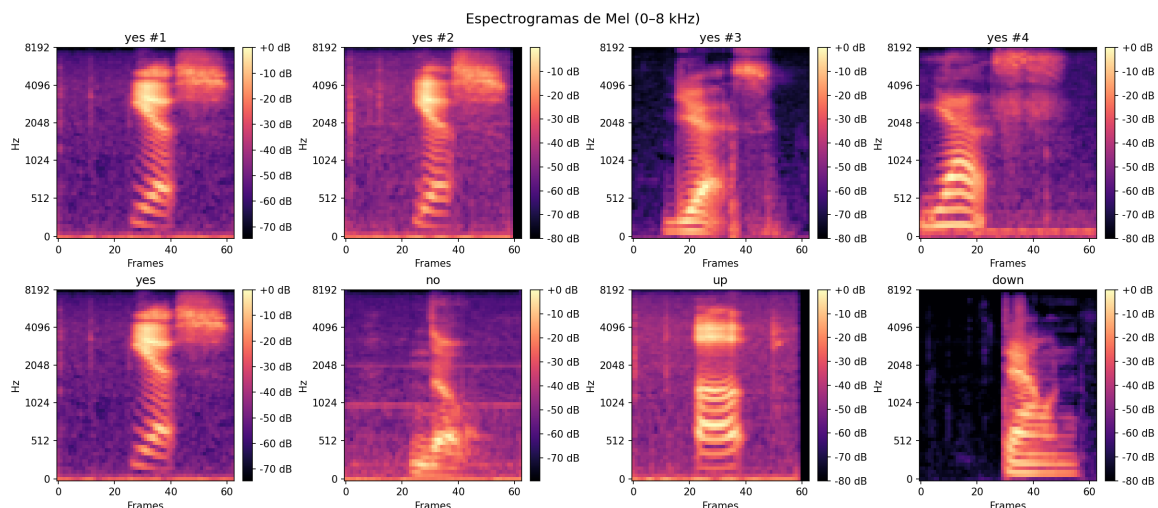


Figura 4: Panel de espectrogramas de Mel utilizado para analizar variabilidad intra-clase y diferencias entre clases.

3.2. Segmentación con ventana deslizante

Antes de pasar el espectrograma a la CNN, se divide en **ventanas temporales con solapamiento**. Cada ventana cubre W columnas del eje temporal y representa una región acústica del audio. El solapamiento garantiza que las transiciones entre sílabas no queden partidas entre dos ventanas sin contexto compartido.

```

1 def extraer_ventanas(mel, window_size=16, hop=8):
2     """
3     mel: array (F, T)
4     window_size: ancho de cada ventana en frames temporales
5     hop: desplazamiento entre ventanas (hop < window_size -> solapamiento)
6
7     Retorna: array (N, F, window_size) con N ventanas
8     """
9     F, T = mel.shape
10    ventanas = []
11    for start in range(0, T - window_size + 1, hop):
12        ventana = mel[:, start : start + window_size] # (F, W)
13        ventanas.append(ventana)
14    return np.stack(ventanas) # (N, F, W)

```

El overlap entre ventanas consecutivas es $\text{window_size} - \text{hop}$. Con $\text{window_size}=16$ y $\text{hop}=8$ el solapamiento es del 50 %, lo que asegura que cada transición fonética aparezca representada en al menos dos ventanas consecutivas.

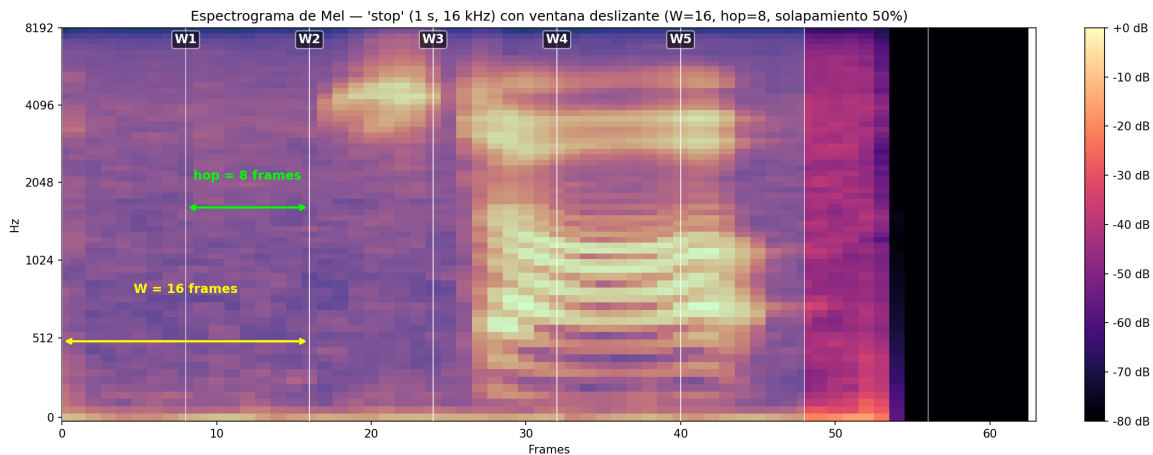


Figura 5: Ejemplo de segmentación de un espectrograma de Mel mediante ventana deslizante con solapamiento.

Nota sobre shapes: El espectrograma tiene shape (F, T) con frecuencia en el eje 0 y tiempo en el eje 1. Cada ventana extraída tiene shape (F, W) . Al pasarla a la CNN como imagen monocanal: $(1, F, W)$. Es importante no confundir los ejes: la CNN debe recibir frecuencia en la dimensión de altura e índice de ventana como paso de secuencia para la RNN.

3.3. Arquitectura CRNN

La CNN se aplica de forma **independiente a cada ventana** (equivalente a TimeDistributed en Keras). Su función es comprimir cada ventana (F, W) en un único vector de ca-

racterísticas que resume los patrones acústicos presentes en ese segmento temporal. La secuencia de esos vectores, ordenada cronológicamente, es la entrada a la RNN.

```

1 import torch
2 import torch.nn as nn
3
4 class CNNEncoder(nn.Module):
5     """Procesa una sola ventana (batch, 1, F, W) -> vector (batch, features)"""
6     def __init__(self, n_mels=64, window_size=16):
7         super().__init__()
8         self.cnn = nn.Sequential(
9             nn.Conv2d(1, 32, kernel_size=(3, 3), padding=1),
10            nn.ReLU(), nn.BatchNorm2d(32),
11            nn.MaxPool2d(2), # -> (32, F/2, W/2)
12
13            nn.Conv2d(32, 64, kernel_size=(3, 3), padding=1),
14            nn.ReLU(), nn.BatchNorm2d(64),
15            nn.MaxPool2d(2), # -> (64, F/4, W/4)
16        )
17        # calcular tamaño de salida aplanada
18        self.out_size = 64 * (n_mels // 4) * (window_size // 4)
19        self.fc = nn.Linear(self.out_size, 128)
20
21    def forward(self, x):
22        # x: (batch, 1, F, W)
23        x = self.cnn(x) # (batch, 64, F/4, W/4)
24        x = x.flatten(1) # (batch, out_size)
25        return self.fc(x) # (batch, 128)
26
27
28 class CRNN(nn.Module):
29     def __init__(self, n_mels=64, window_size=16, n_classes=10,
30                  rnn_hidden=128, rnn_layers=2):
31         super().__init__()
32         self.encoder = CNNEncoder(n_mels, window_size)
33
34         self.rnn = nn.GRU(
35             input_size=128,
36             hidden_size=rnn_hidden,
37             num_layers=rnn_layers,
38             batch_first=True,
39             bidirectional=True,
40             dropout=0.3
41         )
42         self.fc = nn.Linear(rnn_hidden * 2, n_classes) # *2 por
43         bidireccional
44
45     def forward(self, x):
46         # x: (batch, N, 1, F, W) -- N ventanas por muestra
47         batch, N, C, F, W = x.shape
48         x = x.view(batch * N, C, F, W) # procesar todas las
49         # ventanas juntas
50         x = self.encoder(x) # (batch*N, 128)
51         x = x.view(batch, N, -1) # (batch, N, 128) --
52         # restaurar secuencia
53         x, _ = self.rnn(x) # (batch, N, hidden*2)
54         x = x[:, -1, :] # ultimo estado -> (batch,

```

```

52         hidden*2)
        return self.fc(x)

```

Punto de partida: La arquitectura anterior es una sugerencia. Se espera que optimicen tanto la CNN como la RNN para obtener la mayor precisión posible. Ver sección 3.5 para los experimentos requeridos.

3.4. Entrenamiento

- Usar `CrossEntropyLoss` y el optimizador Adam.
- Implementar un scheduler de learning rate (ej. `ReduceLROnPlateau`).
- **División del dataset:** El dataset provee archivos `validation_list.txt` y `testing_list.txt` con particiones oficiales que garantizan **separación estricta por hablante**: ningún hablante del conjunto de test aparece en entrenamiento. Respetar estas particiones para que los resultados sean comparables.
- Guardar checkpoints periódicamente.

3.5. Experimentos

El objetivo es optimizar la CRNN para obtener la mayor accuracy posible en el conjunto de test. Documentar al menos **3 experimentos** en una tabla comparativa, explorando las siguientes dimensiones:

CNN (encoder de ventanas)

- Número de bloques convolucionales (profundidad).
- Número de filtros por capa.
- Tamaño de kernel.
- Dropout dentro del encoder.

Ventana deslizante

- Tamaño de ventana W (ventanas más anchas dan más contexto local a la CNN; más estrechas producen secuencias más largas para la RNN).
- Solapamiento (hop): mayor overlap $\rightarrow$ secuencia más larga, más redundancia entre tokens.

RNN

- Tipo de celda: LSTM vs. GRU.
- Número de capas y unidades ocultas.
- Bidireccional vs. unidireccional.
- Estrategia de agregación: último estado oculto vs. average pooling sobre todos los estados.

Experimento	CNN	Ventana (W, hop)	RNN	Acc. Val	Acc. Test	Observaciones
E1 — Línea base	2 bloques, 32/64 filtros	16, 8	GRU 2 capas, 128 uni.			
E2	...	...	...			
E3	...	...	...			

3.6. Qué reportar

- Curvas de loss y accuracy (train vs. validación) por época para cada experimento.
- Tabla comparativa de todos los experimentos (sección 3.5) con accuracy en validación y test.
- **Matriz de confusión** del mejor modelo sobre el conjunto de test.
- Análisis de los espectrogramas según lo indicado en la sección 3.1.
- Análisis: ¿qué comandos se confunden más? ¿Tiene sentido fonéticamente?
- Al menos 5 ejemplos de predicciones con el espectrograma, la etiqueta real y la predicción del modelo.

4. Modalidad de Entrega y Evaluación

El trabajo debe realizarse de manera grupal con exactamente **3 integrantes** (o 4 en caso de que la cantidad de estudiantes del curso no sea múltiplo de 3). Todo el material deberá almacenarse en un repositorio **GitHub privado**, al cual deberán agregar a los docentes como colaboradores.

Fecha límite de entrega: Viernes 5 de junio, hasta las 23:59 hs. No se aceptará ningún trabajo entregado con posterioridad a dicha fecha y hora.

La entrega se realizará mediante un **correo electrónico con copia a ambos docentes**, con el siguiente formato:

- **Asunto:** AAIL - Entrega TPN°2 - Grupo Y (reemplazar Y por el número del grupo).
- **Cuerpo:** debe incluir:
 1. Número de grupo.
 2. Nombre completo de cada integrante.
 3. Link al repositorio de GitHub.

4.1. Notebooks

Cada ejercicio debe resolverse en un **Jupyter Notebook separado**. El notebook debe cumplir los siguientes requisitos:

- El código debe organizarse en celdas según su propósito, con salidas claras (prints y visualizaciones) que documenten el resultado de cada proceso.

- Cada celda de código debe ir precedida de una celda Markdown con: **título** (usando jerarquía #, ##, ###), una breve descripción de lo que se realizará y las decisiones tomadas con su justificación.
- Cada celda de código debe ir seguida de una celda Markdown que explique el resultado obtenido y las conclusiones derivadas.
- El código debe incluir comentarios explicativos sobre conjuntos de líneas con un mismo propósito. Se deben usar nombres de variables descriptivos en notación `snake_case`.

4.2. Informe

El repositorio deberá contener un **informe en formato PDF** que incluya:

- Descripción del procedimiento seguido para resolver cada ejercicio.
- Decisiones de diseño tomadas y sus justificaciones.
- Desafíos encontrados durante el desarrollo.
- Conclusiones sobre los aprendizajes obtenidos.

4.3. Defensa

La defensa se realizará de manera **virtual** en dos fechas consecutivas. La fecha y hora de cada grupo será sorteada y comunicada con anticipación. El grupo deberá conectarse al enlace de videoconferencia en el horario asignado. **El integrante que no se presente a la defensa desaprobará el trabajo completo.** De no poder asistir en la fecha u horario sorteado, se puede negociar con otro grupo para intercambiar lugares.

La defensa consta de tres fases:

1. **Exposición grupal** (máximo 15 minutos): el grupo deberá presentar el desarrollo de las resoluciones a los ejercicios, las decisiones tomadas en cada uno y sus resultados. Se evaluará la capacidad de seleccionar los aspectos más relevantes y la gestión del tiempo. Se recomienda distribuir la exposición equitativamente entre los integrantes.
2. **Preguntas conceptuales individuales**: cada integrante responderá preguntas de carácter conceptual para terminar de definir la nota.
3. **Devolución general** por parte de los docentes.